SAIL-PROXY User Guide - Chapter 14

st-gr

2026-09-15

SAIL-PROXY User Guide

Multi-provider AI Gateway for SAP AI Core **Author:** *st-gr*

<< Previous Chapter | Content Table | Next Chapter >>

Using the OpenAI Realtime API

SAIL-PROXY relays the OpenAI **Realtime API** — the WebSocket protocol for low-latency text and speech conversations — to SAP AI Core's `gpt-realtime` deployment. Any server-side client that speaks the Realtime protocol connects to the gateway instead of OpenAI, authenticates with a gateway API key, and every response it receives is metered and counted against the same quotas as any other request.

Prerequisites

- **SAIL-PROXY installed and running** (see Installation) and a **gateway API key** (see Admin Cockpit).
- A **running `gpt-realtime` deployment** in your SAP AI Core resource group. It appears as `gpt-realtime--deployed` in the Model Library and in `GET /v1/models`; the bare `gpt-realtime` name is accepted too and resolves to the same deployment.
- A **server-side client**. The connection carries the API key in a header, which browsers cannot set; browser clients are not supported yet.

Connecting

Setup	WebSocket URL
Local gateway	`ws://localhost:3000/openai/v1/realtime`
Docker	`wss://<your-host>/gateway/openai/v1/realtime`
Kyma	`wss://<your-host>/gateway/openai/v1/realtime`

Send the key as `Authorization: Bearer <gateway API key>` (or `x-api-key: <key>`). The `model` query parameter is optional and defaults to `gpt-realtime`. The handshake is refused with a normal HTTP status when something is wrong: `401` (key), `403` (the model is not in your entitlement catalog), `429` (over quota, with a `Retry-After` header), `404` (no running realtime deployment), `502`/`503` (SAP AI Core refused the connection).

After the handshake the session is the standard Realtime protocol: the first event you receive is `session.created`; send `session.update`, `conversation.item.create`, `response.create`, `input_audio_buffer.append` and the rest as documented by OpenAI. Text and audio both work.

Node.js

```
const WebSocket = require('ws');
const ws = new WebSocket('ws://localhost:3000/openai/v1/realtime?model=gpt-realtime', {
  headers: { Authorization: `Bearer ${process.env.SAILPROXY_API_KEY}` },
});
ws.on('open', () => {
  ws.send(JSON.stringify({ type: 'session.update', session: { type: 'realtime', output_modali
        ties: ['text'] } }));
  ws.send(JSON.stringify({ type: 'conversation.item.create', item: { type: 'message', role:
        'user', content: [{ type: 'input_text', text: 'Say hello.' }] } }));
  ws.send(JSON.stringify({ type: 'response.create' }));
});
ws.on('message', (data) => {
  const event = JSON.parse(data);
  if (event.type === 'response.output_text.delta') process.stdout.write(event.delta);
  if (event.type === 'response.done') { console.log('\nusage:', event.response.usage); ws.clo
        se(); }
});
```

Python

```
import asyncio, json, os, websockets

async def main():
    url = "ws://localhost:3000/openai/v1/realtime?model=gpt-realtime"
    headers = {"Authorization": f"Bearer {os.environ['SAILPROXY_API_KEY']}"}
    async with websockets.connect(url, additional_headers=headers) as ws:
        await ws.send(json.dumps({"type": "session.update", "session": {"type": "realtime",
        "output_modalities": ["text"]}}))
        await ws.send(json.dumps({"type": "conversation.item.create", "item": {"type": "messa
        ge", "role": "user", "content": [{"type": "input_text", "text": "Say hello."}]}}))
        await ws.send(json.dumps({"type": "response.create"}))
        async for message in ws:
            event = json.loads(message)
            if event["type"] == "response.output_text.delta":
                print(event["delta"], end="", flush=True)
            if event["type"] == "response.done":
                print("\nusage:", event["response"]["usage"])
                break

asyncio.run(main())
```

OpenAI SDKs

The official OpenAI SDKs ship a realtime client. It builds its endpoint as `<base URL>/realtime?model=…` and always connects with `wss://`, so it works against a gateway behind a TLS ingress — Docker or Kyma, with the base URL `https://<your-host>/gateway/openai/v1` and your gateway API key — but it cannot reach a plain `ws://` local gateway. Use the Node or Python sample above for local development.

```
import OpenAI from 'openai';
import { OpenAIRealtimeWS } from 'openai/realtime/ws';

const client = new OpenAI({ apiKey: process.env.SAILPROXY_API_KEY, baseURL: 'https://<your-ho
        st>/gateway/openai/v1' });
const rt = await OpenAIRealtimeWS.create(client, { model: 'gpt-realtime' });
rt.on('response.output_text.delta', (event) => process.stdout.write(event.delta));
```

Metering and quotas

- Opening a session counts as one request, and **every response** the model produces counts as one more — including responses the server starts itself when voice activity detection is on.

- The tokens of each response are recorded when it completes and show up in the Admin Cockpit like any other usage. Audio tokens are counted separately from text tokens and priced at the model's audio rates once an administrator has entered them in the Model Library — SAP publishes them in SAP Note 3437766, not through the catalogue. Until then, audio tokens are priced at the text rates, which understates the cost of voice sessions. Cached input tokens, whether text or audio, are priced at the model's cache-read rate (or its input rate when no cache-read rate is maintained), so a voice session that replays earlier audio is priced below the audio rate for that share until SAP confirms how cached audio is billed.
- If a limit is reached during a session, you receive an `error` event whose `error.type` is `quota_exceeded` (with the window, the limit and when it resets), the running response is cancelled, and the session closes with WebSocket code `1008` and reason `quota_exceeded`. A key revoked or a user deactivated mid-session closes it with `1008` and reason `unauthorized`.
- If SAP AI Core drops the connection, the session closes with code `1011` and reason `upstream_error`.
- When the gateway restarts, open sessions close with code `1001` and reason `server_shutdown`; reconnect and start a new session.

Limits

- Text and audio in both directions are supported; **WebRTC and SIP are not** (SAP AI Core offers only the WebSocket transport).
- No browser clients yet: the ephemeral client secrets the browser flow needs are not issued by the gateway.
- Data masking (pseudonymization) is not applied to realtime sessions.

For general troubleshooting, see the Troubleshooting guide or the FAQ.